

NetSpect Backend Core - Project Report

Project Overview

NetSpect is a Spring Boot-based backend engine designed for Security Operations Centers (SOC). It provides real-time event processing, storage, and automated threat response capabilities. The system ingests security events from various sources, processes them through Kafka streams, stores them in Elasticsearch for search and PostgreSQL for persistence, and delivers real-time alerts via WebSocket to dashboards.

Key Features

- **Event Streaming:** Kafka-based event ingestion from agents and SIEM systems
- **Multi-Storage:** Dual storage in Elasticsearch (search/analytics) and PostgreSQL (persistence/audit)
- **Real-time Alerts:** WebSocket push notifications to live dashboards
- **SOAR Integration:** Automated threat response and command execution
- **RESTful APIs:** Query and management endpoints
- **Event Types:** Support for Login, Network, and Endpoint security events

Technology Stack

Component	Technology	Version
Framework	Spring Boot	3.5.13
Language	Java	17
Build Tool	Maven	3.x (via wrapper)
Message Broker	Apache Kafka	3.7
Search Engine	Elasticsearch	8.x
Database	PostgreSQL	16
Real-time	Spring WebSocket (STOMP)	-
HTTP Client	Spring WebFlux (WebClient)	-
Testing	JUnit, Spring Boot Test	-
Documentation	Maven Site Plugin	-

Architecture

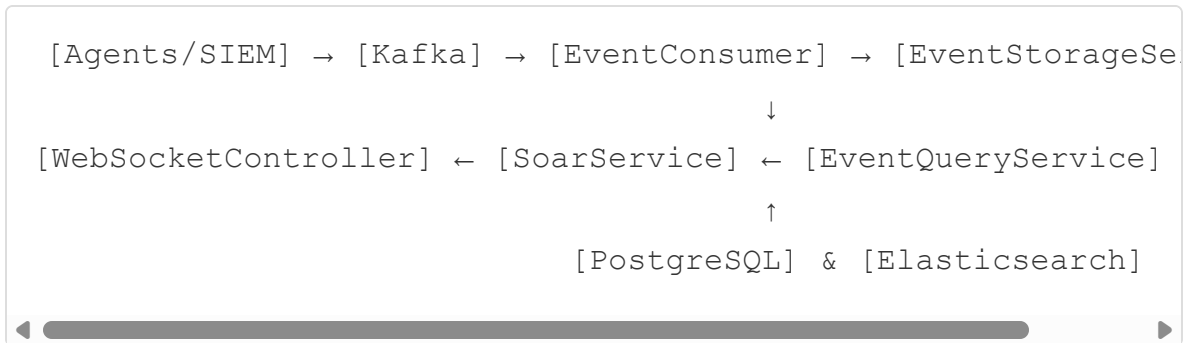
The application follows a layered architecture with clear separation of concerns:

Data Flow

1. **Ingestion:** Events arrive via Kafka topics (`events.network` , `events.endpoint` , `events.alerts`)
2. **Processing:** EventConsumer processes incoming messages
3. **Storage:** EventStorageService writes to both Elasticsearch and PostgreSQL
4. **Analysis:** EventQueryService provides search and analytics
5. **Response:** SoarService executes automated responses

6. **Notification:** WebSocketController pushes real-time alerts

Component Diagram



Code Structure

Main Packages

- `com.soc.backend_core` : Main application and configuration
- `com.soc.backend_core.controller` : REST controllers and WebSocket handlers
- `com.soc.backend_core.service` : Business logic services
- `com.soc.backend_core.consumer` : Kafka message consumers
- `com.soc.backend_core.producer` : Kafka message producers
- `com.soc.backend_core.repository` : Data access layers (JPA and Elasticsearch)
- `com.soc.backend_core.Entities` : Domain models and entities
- `com.soc.backend_core.dto` : Data transfer objects

Key Classes

Configuration

- `BackendCoreApplication` : Main Spring Boot application class
- `KafkaConfig` : Kafka producer/consumer and topic configuration
- `WebSocketConfig` : STOMP WebSocket message broker setup

Controllers

- `EventController` : REST endpoints for event management
- `EventQueryController` : Search and query APIs
- `SoarController` : SOAR command endpoints
- `WebSocketController` : WebSocket message handling

Services

- `EventStorageService` : Handles data persistence to both stores
- `EventQueryService` : Provides search and analytics functionality
- `SoarService` : Manages automated threat responses

Entities

- `UnifiedEvent` : Core event domain model
- `AlertMessage` : Alert notification model
- `SoarCommand` : Automated response command model
- `EventRecord` : JPA entity for PostgreSQL
- `EventDocument` : Elasticsearch document model

DTOs

- `LoginEventRequest` : Login event data
- `NetworkEventRequest` : Network event data
- `EndpointEventRequest` : Endpoint event data
- `SoarCommandRequest` : SOAR command data

Dependencies

Core Dependencies

- Spring Boot Starters: web, validation, data-jpa, data-elasticsearch, websocket, webflux
- Spring Kafka: kafka, kafka-test

- Database: postgresql
- JSON Processing: jackson-datatype-jsr310
- Utilities: lombok

Test Dependencies

- spring-boot-starter-test
- spring-kafka-test

Build Configuration

Maven Plugins

- `spring-boot-maven-plugin` : Boot application packaging
- `maven-compiler-plugin` : Java compilation with Lombok annotation processing
- `maven-javadoc-plugin` : Documentation generation

Java Version

- Source/Target: Java 17
- Annotation Processing: Lombok enabled

Configuration

Application Properties

Located in `src/main/resources/application.yml`

Key configurations include: - Kafka broker settings - Elasticsearch cluster configuration - PostgreSQL database connection - WebSocket endpoint settings - Logging levels

Testing

Test Structure

- `BackendCoreApplicationTests` : Main application context test
- Integration tests for Kafka consumers/producers
- Unit tests for services and utilities

Test Execution

```
./mvnw test
```

Deployment

Local Development

1. Ensure Java 17+ is installed
2. Start required services (Kafka, Elasticsearch, PostgreSQL)
3. Run `./mvnw spring-boot:run`

Production Build

```
./mvnw clean package  
java -jar target/backend-core-0.0.1-SNAPSHOT.jar
```

Docker Support

Docker Compose configuration available for local development environment.

API Endpoints

REST APIs

- `GET /api/events` : Query events with filters
- `POST /api/events` : Create new events
- `GET /api/events/{id}` : Get specific event
- `POST /api/soar/commands` : Execute SOAR commands

WebSocket

- Endpoint: `/ws/alerts`
- Subscribe: `/topic/alerts` for real-time alerts
- Send: `/app/alert` to broadcast alerts

Security Considerations

- CORS configuration for WebSocket endpoints
- Input validation using Bean Validation
- Secure communication protocols (HTTPS, WSS)
- Audit trail in PostgreSQL
- Role-based access control (to be implemented)

Performance Characteristics

- Sub-second latency for event processing via Kafka
- Scalable storage with Elasticsearch for search
- Real-time WebSocket push notifications
- Concurrent Kafka listeners for high throughput

Future Enhancements

- Authentication and authorization

- Event correlation and pattern detection
- Advanced SOAR workflows
- Metrics and monitoring integration
- Multi-tenancy support
- API rate limiting and throttling

Development Notes

- Uses Lombok for boilerplate reduction
- Follows Spring Boot conventions
- Maven wrapper for consistent builds
- Javadoc generation enabled
- Test coverage to be expanded

Conclusion

NetSpect Backend Core provides a solid foundation for SOC operations with modern technologies and scalable architecture. The modular design allows for easy extension and customization based on specific security requirements.